



Experiment – 1.3.2

Student Name: Damandeep Singh

UID: 24BDA7O313

Branch: CSE (Data Science)

Section/Group: 24 BDS-4(B)

Semester: 5th

Date of Performance: 22-07-2026

Subject: Full Stack II

Subject Code: 24CSP-337

1. Aim:

To implement role-based access control and secure application routes based on user permissions.

2. Software Required:

- IDE/Environment: Visual Studio Code
- Runtime: Node.js (v18 or above), npm
- Backend: Express.js, jsonwebtoken
- Frontend: React 18, Vite, Redux Toolkit, react-redux, react-router-dom
- Browser: Google Chrome (for testing and rendering)

3. Objective:

To implement role-based access control and secure application routes based on user permissions, so that different user roles (admin, editor, viewer) see and can access different parts of the application, both in the UI and at the API level.

4. Theory:

Role-based access control embeds a role claim inside the authenticated user's JWT and checks it against an allow-list at two independent layers. On the server, middleware (requireRole) rejects requests from users whose role is not in the allow-list with a 403 Forbidden response. On the client, route guards check the same role before rendering a page, redirecting unauthorized users to a 403 page before they ever see restricted content. Relying on only one of these layers is insufficient: client-only checks can be bypassed by calling the API directly.



5. Procedure/Algorithm:

- Step 1: Assign each user a role (admin, editor, viewer) and embed it in their JWT payload at login.
- Step 2: Write a requireRole(...allowedRoles) middleware factory that checks req.user.role against the allow-list.
- Step 3: Apply requireRole to specific backend routes (e.g. GET /api/admin/users → admin only).
- Step 4: Build a ProtectedRoute component that redirects unauthenticated users to /login, and users lacking a required role to /unauthorized.
- Step 5: Wire routes so /dashboard is open to all roles, /editor requires admin or editor, and /admin requires admin only.
- Step 6: Conditionally render navigation links and page content (e.g. a read-only Dashboard for viewers) based on role.

6. Code and Output:

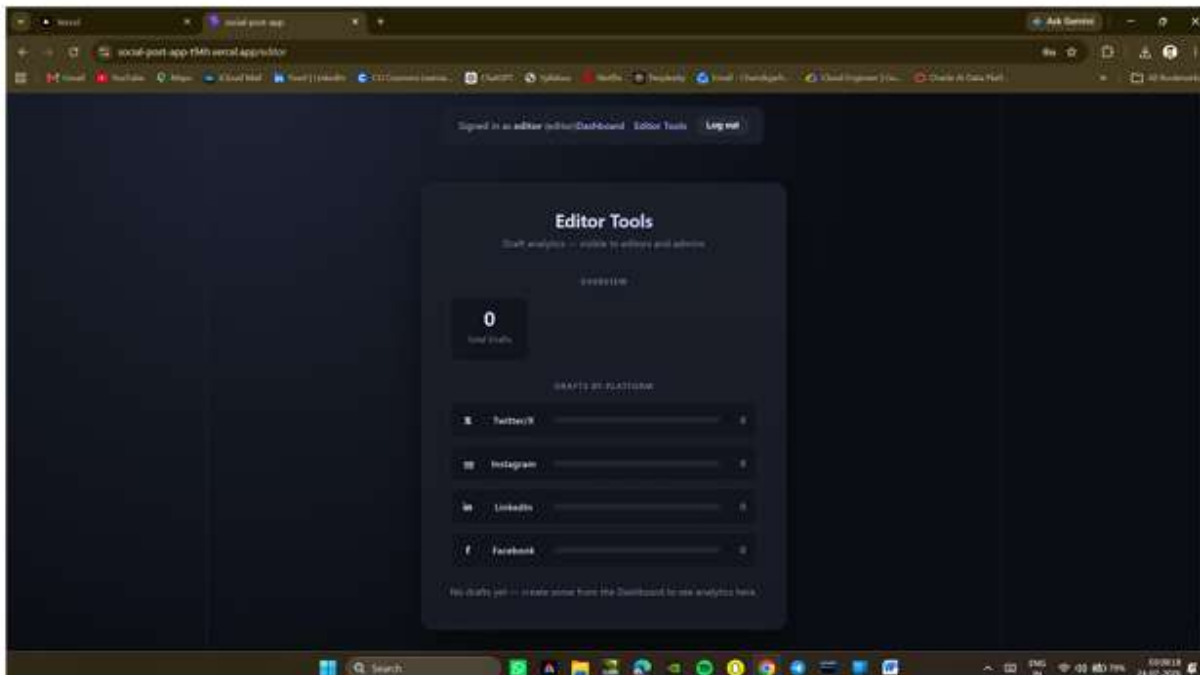
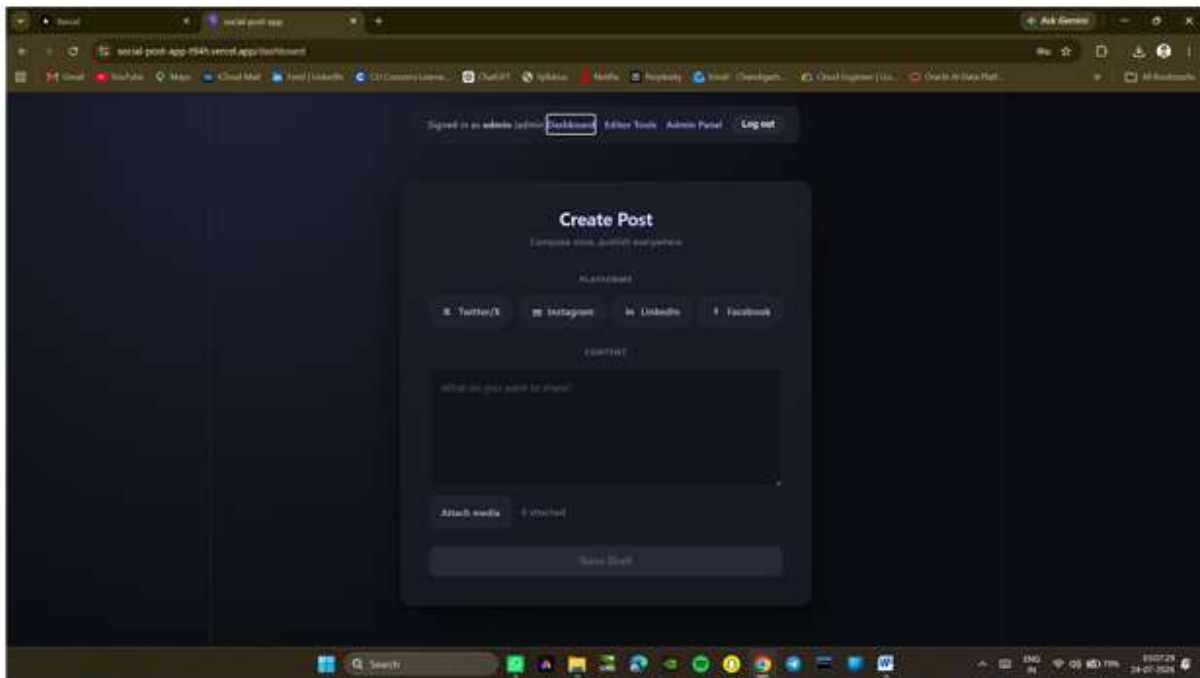
```
function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Forbidden: insufficient role' });
    }
    next();
  };
}

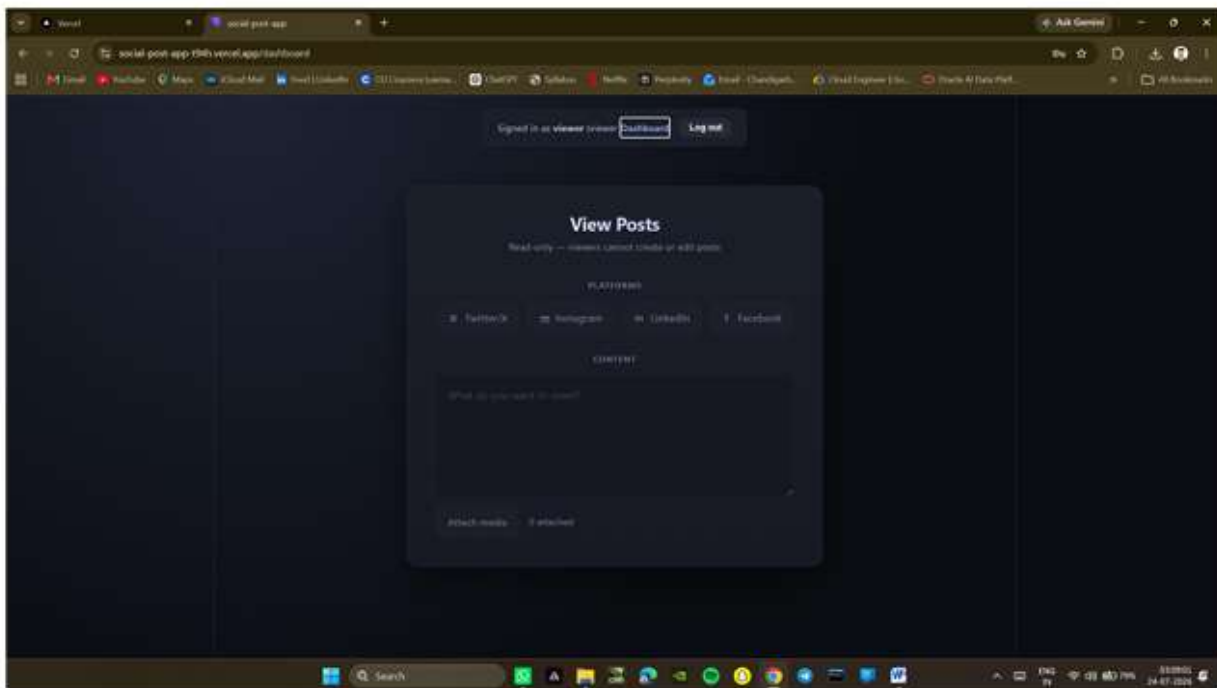
export default function ProtectedRoute({ roles }) {
  const isAuthenticated = useSelector(selectIsAuthenticated);
  const hasRole = useSelector((state) => (roles ? selectHasRole(state, roles) : true));
  if (!isAuthenticated) return <Navigate to='/login' replace />;
  if (roles && !hasRole) return <Navigate to='/unauthorized' replace />;
  return <Outlet />;
}
```



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Discover. Learn. Empower.





7. Conclusion:

Role-based access control was successfully layered on top of the authentication system: admin, editor, and viewer accounts each see a different navigation menu and have different route access, enforced both by server-side middleware and by client-side route guards. Testing across all three roles confirmed correct enforcement at both layers.

8. Learning Outcomes:

- Learned to write reusable Express middleware for role-based authorization.
- Learned to enforce access control at two independent layers — server API and client routing.
- Understood how to build memoized, parameterized selectors for role checks.